ROB 498/599 · 3D Robot Perception · Fall 2026

L02 The camera matrix, and rigid-body state

Everything in this course is a pose, a measurement, or an uncertainty about one of them.

Wed Sep 2, 2026 • WRITE-IN build

The camera matrix, and rigid-body state

L02 · Unit 0

Everything in this course is a pose, a measurement, or an uncertainty about one of them.

Monday ran to *a point is a ray*. The first half of today finishes the camera: we assemble $P = K[R \mid t]$ and calibrate a real lens. The second half asks what kind of object $[R \mid t]$ is.

Two stages, and that is the whole camera

- **Extrinsics**: a rigid motion. Where the camera is, and which way it points. Six numbers, and they change every frame.
- **Intrinsics**: the projection, then metres to pixels. What the camera *is*. Five numbers, fixed until someone changes the lens.

Build each stage on its own, then multiply. That product is P.

The projection: camera frame to image

the pinhole ratio, in coordinates

Monday's similar triangles in coordinates, then rewritten up to scale so the division becomes a matrix:

$$\begin{pmatrix} x_i \\ y_i \end{pmatrix} = \frac{f}{z_c} \begin{pmatrix} x_c \\ y_c \end{pmatrix} \quad \Longrightarrow \quad \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} \sim \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

The depth you divided by is the scale factor you threw away.

Projections

Careful

"Projection" names a linear map between vector spaces. The matrix above is the projection from the 3D camera frame to the 2D image. It is a different object from the projection matrix P.

Intrinsics: from metres to pixels

(x_i, y_i) is in metres on the sensor, measured from the optical axis. A pixel index is in pixels, counted from a corner. Three facts close the gap.

1. A pixel has a physical size (p_x, p_y), so $f_x = F / p_x$ and $f_y = F / p_y$ convert the focal length F from millimetres into pixels.
2. The two origins differ. Shift by the **principal point** (c_x, c_y), the pixel where the optical axis pierces the sensor.
3. If the sensor axes are not perpendicular there is a skew $s = f_x \tan \alpha$. It is 0 on every sensor you will meet.

$$u = f_x \frac{X}{Z} + c_x, \qquad v = f_y \frac{Y}{Z} + c_y$$

The intrinsic matrix

$$\mathrm{K} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Five degrees of freedom, all in pixels.

f_x	focal length in pixels, F / p_x
f_y	focal length in pixels, F / p_y
c_x	principal point, horizontal
c_y	principal point, vertical
s	skew, $f_x \tan \alpha$; zero in practice

K is upper triangular and invertible, so $\mathrm{K}^{-1}\tilde{\mathrm{u}}$ takes a pixel back to a ray direction. That inverse is how every depth sensor in L04 gets used.

A camera, with numbers

Build K for a real sensor, then project one point

Everything else today is this arithmetic with the unknowns moved around.

The principal point

and why it is not the image centre

Look straight out along the lens axis. The pixel you land on is (c_x, c_y): the origin every projection is measured from, and the centre radial distortion grows away from. Lens and sensor are aligned to a tolerance, so it lands off the middle of the array.

Sensor rectangle: pixel origin, optical axis, image centre

Assuming the centre is a bias

Trap

Setting $c_x = W/2$ is a **bias**, not an approximation. In the camera two slides ago the measured c_x was 642.1 and the image centre is 640: two pixels, applied to every observation, in the same direction, forever.

A bias does not average out with more data. It bends the map, and the residual stays clean while it does.

OpenCV returns c_x and c_y from calibration for this reason. Use the values it gives you rather than the image dimensions.

The extrinsics: world frame to camera frame

$$X_c = R X_w + t$$

A rotation and a translation, in ordinary Euclidean terms.

$$\begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} R & t \\ 0^\top & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

The bottom row is the reason for the fourth coordinate: it turns *add a vector* into *multiply by a matrix*, so the whole chain composes.

A pose, with numbers

World frame x forward, y left, z up. The camera sits at $\mathrm{C} = (1,\ 0,\ 0.5)$ looking down world $+x$, optical frame z forward, x right, y down. A point five metres down the corridor: $\mathrm{X}_w = (5,\ -0.5,\ 0.7)$.

Rows of R are the camera axes in world coordinates. Then move the point.

t is **not** where the camera is. The camera sits at the origin of its own frame, so C must map to zero: $0 = \mathrm{RC} + \mathrm{t}$. Equivalently, t is where the *world* origin lands in camera coordinates.

Compose the two stages

K turns the flattened point into pixels, so the chain is one product.

$$\tilde{x} \sim \underset{3\times 3}{K} \underset{3\times 4}{[I \mid 0]} \underset{4\times 4}{\begin{bmatrix} R & t \\ 0^\top & 1 \end{bmatrix}} \tilde{X}_w \qquad \Longrightarrow \qquad \boxed{P = K[R \mid t]}$$

$w\,\tilde{x} = P\,\tilde{X}_w$: five intrinsic numbers, six extrinsic ones, one overall scale that does not matter.

The camera matrix

- Eleven degrees of freedom: twelve entries, minus the overall scale.
- The **null space** of P is the camera centre in world coordinates: $\mathrm{P}\tilde{\mathrm{C}} = 0$.
- It maps every world point to a pixel, and it is **not invertible**. A pixel is a ray, not a point.

Estimating P from correspondences

Given known 3D points X_i and the pixels they landed on, solve for P. Writing $\mathsf{p}_1^\top, \mathsf{p}_2^\top, \mathsf{p}_3^\top$ for its rows, clearing the denominator gives two **linear** equations per point:

$$\mathsf{p}_1^\top \tilde{\mathsf{X}}_i - u_i\, \mathsf{p}_3^\top \tilde{\mathsf{X}}_i = 0, \quad \mathsf{p}_2^\top \tilde{\mathsf{X}}_i - v_i\, \mathsf{p}_3^\top \tilde{\mathsf{X}}_i = 0$$

Unknowns, equations per measurement, minimum measurements

Reprojection error

Six correspondences pin P down exactly. A seventh will not fit, because the data is noisy and the correspondences came from an algorithm with its own errors. What you want is the P that misses by least.

$$e = \frac{1}{N} \sum_{i=1}^{N} \left\| \hat{\mathrm{u}}_i - \mathrm{u}_i \right\|_2, \qquad \hat{\mathrm{u}}_i = \pi\left(\mathrm{P}\tilde{\mathrm{X}}_i\right)$$

- e is the mean miss over the N correspondences, and π is the perspective divide, $(a, b, c) \mapsto (a/c,\ b/c)$.
- $\hat{\mathrm{u}}_i$ is where your estimate says point i should have landed; u_i is where it did.
- The units are **pixels**, which is why this is the currency of the whole field.
- A high residual often means a wrong correspondence rather than a bad P.

Where the linear model stops

real lenses bend rays

Correct distortion in **normalised** coordinates $(x, y) = (X/Z,\ Y/Z)$, *before* applying K. Write $r^2 = x^2 + y^2$.

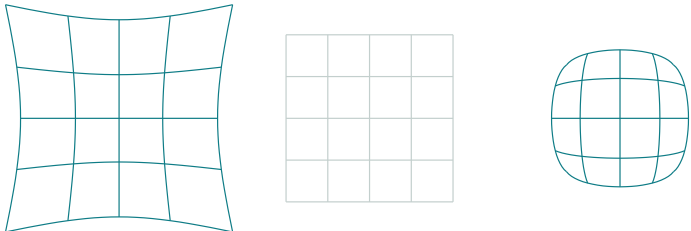
$$x_{\mathrm{d}} = x\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right), \qquad y_{\mathrm{d}} = y\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right)$$

$$x_{\mathrm{d}} = x + \left[2p_1 xy + p_2\left(r^2 + 2x^2\right)\right], \qquad y_{\mathrm{d}} = y + \left[p_1\left(r^2 + 2y^2\right) + 2p_2 xy\right]$$

(x, y) where the ideal lens would put the point, $(x_{\mathrm{d}}, y_{\mathrm{d}})$ where the real one does, r the distance between the point and the optical axis. Radial on the top row, with k_1, k_2, k_3; tangential on the bottom, with p_1, p_2.

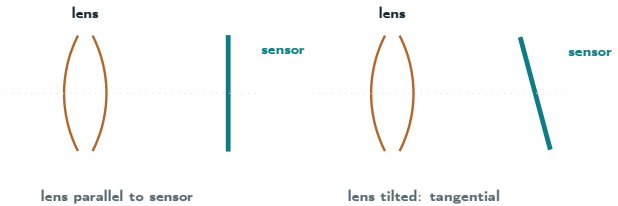
Two radial coefficients are usually enough; k_3 is for wide angle. The model is no longer a matrix.

What radial distortion looks like



$k_1 > 0$ pushes points outward, $k_1 < 0$ pulls them in. Straight lines photograph curved, and the effect grows with r, which is exactly where your calibration target's corners are.

Tangential distortion is a mounting error



It appears when the lens and the sensor are not parallel, which makes it a manufacturing tolerance rather than an optical property.

Trap

Distortion is applied *before* K. Getting the order backwards fits well at the centre and fails at the edges, which is where all your target corners are.

Calibration is an estimation problem

Photograph a planar checkerboard at several orientations. Every photo hands you the same thing you already know how to use: 3D points whose positions you know, and the pixels they landed on.

- **Unknown once:** K and the distortion coefficients k_1, k_2, k_3, p_1, p_2.
- **Unknown per photo:** the pose $[\mathrm{R} \mid \mathrm{t}]$ of the board.
- **Known:** the board geometry, and every detected corner.

Choose all of them together to minimise the reprojection error, summed over every corner in every photo. Nothing new has been introduced except bookkeeping.

> *A calibration reported without its residual is a number somebody made up.*

Half time

The camera is finished. $P = K[R \mid t]$, estimated from correspondences, refined against reprojection error, with a distortion model bolted on the front.

> *So far* $[R \mid t]$ *has been a block of twelve numbers we multiplied by. The rest of the hour asks what kind of object it is.*

Reference for everything above: the MathWorks calibration page, linked under **Resources**. Its notation is the notation on these slides.

Why not just use three numbers?

An orientation has three degrees of freedom, so store it as three numbers, roll, pitch and yaw, and then use the tools you already own. Add two of them. Average a batch. Differentiate. Run least squares.

Every one of those goes wrong. At pitch $\pm 90^\circ$ roll and yaw collapse onto the same axis and a degree of freedom disappears. Rotations do not commute, so three numbers cannot record which order you meant, and there are twelve Euler conventions with no default. The componentwise average of two orientations is not an orientation.

Reaching for a different representation does not rescue it. Quaternions trade the singularity for a sign ambiguity, and rotation matrices trade it for six constraints you now have to maintain by hand.

> *The labels were never the problem. Rotations are not a set you can do arithmetic on.*

So we need two things

- A statement of which operations are **legal** here, and a guarantee they stay inside the set. That is what calling $\mathrm{SO}(3)$ a **group** buys.
- A way to take derivatives, since every optimiser and every filter you own is built on *add a small correction*. That is what the **tangent space** buys.

Neither is abstraction for its own sake. Each one answers a question the last slide raised, and the rest of the hour is those two answers and their consequences.

What a group is

A **group** is a set of objects together with one way of combining two of them, obeying four rules.

- **Closure.** Combine two members and you get a member, never anything else.
- **Associativity.** $(ab)c = a(bc)$, so a chain needs no brackets.
- **Identity.** One member e that changes nothing: $ae = ea = a$.
- **Inverse.** Every member a has a partner a^{-1} with $aa^{-1} = e$.

Set and operation	Group?	Why
Integers, $+$	Yes	identity 0, inverse $-a$
Integers, $\times$	No	2 has no integer inverse
Shuffles of a deck	Yes	undo any shuffle, do nothing = identity
Rotations, composition	Yes	the one we need

$\mathrm{SO}(3)$: rotations

$$\mathrm{SO}(3) = \left\{ \mathrm{R} \in \mathbb{R}^{3\times 3} \;:\; \mathrm{R}^{\top}\mathrm{R} = \mathrm{I},\ \det \mathrm{R} = +1 \right\}$$

Count the degrees of freedom: 9 entries, how many constraints?

Three numbers of information, stored in nine, held on a curved surface.

$\mathrm{SE}(3)$: poses

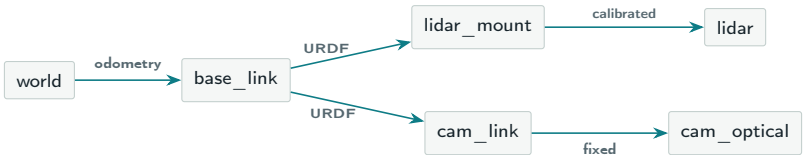
$$T = \begin{bmatrix} R & t \\ 0^\top & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4}, \qquad R \in \mathrm{SO}(3),\ t \in \mathbb{R}^3$$

- Six degrees of freedom: three from R, three from t.
- Composition is matrix multiplication, which is why we padded points to four components.
- Inverse in closed form: $T^{-1} = \begin{bmatrix} R^\top & -R^\top t \\ 0^\top & 1 \end{bmatrix}$.

Trap

Never call `np.linalg.inv` on a pose. It is slower, and it silently drifts off the manifold.

Frames and transform trees



- Each edge is a pose. Reading a chain means multiplying along the path: $T_{wc} = T_{wb} T_{bc}$.
- It is a **tree**: exactly one path between any two frames, so a lookup is unambiguous.

Notation: T_{wc} maps points **from** the camera **into** the world, $p_w = T_{wc}\, p_c$. Adjacent subscripts cancel.

The four edges fail in four different ways

Edge	Where the number comes from	What kind of error it has
world → base	Odometry or SLAM	Grows without limit until a loop closes.
base → mount	A CAD model, via the URDF	Fixed and wrong, if the part was fitted by hand.
mount → lidar	You estimated it	Fixed and uncertain. Bends the whole map.
cam → optical	A convention someone chose	Either exactly right or an axis flip.

Only the first one changes over time. The other three are constants, which makes them easy to forget and impossible to notice.

You cannot add two rotations

You want to say: *my estimate is $\bar{R}$, and the truth is a little off*. You cannot write $R = \bar{R} + \Delta R$, because the sum is not a rotation.

Average a 0° and a 180° rotation about z

What a vector space gives you

A **vector space** is a set where you can add any two members, scale any member, and stay inside. $\mathbb{R}^3$ is one; $\mathrm{SO}(3)$ is not. Calculus is built from those two operations:

$$\frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x + h\mathrm{e}) - f(x)}{h}$$

- f is whatever you are differentiating, e a direction to move in and h a small step along it. Forming $x + h\mathrm{e}$ needs addition *and* scaling, and the result has to remain a legal input to f.
- On $\mathrm{SO}(3)$ that step leaves the set, so the limit is asking about a point that does not exist.
- Gradient descent, Gauss–Newton and Kalman updates all *add a correction*. None can run here.

> *No addition, no derivative. No derivative, no optimiser.*

Flatten it locally: the tangent space

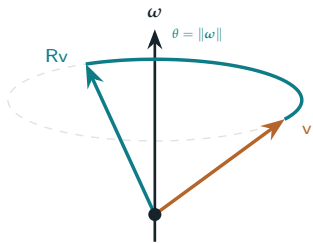
$\mathrm{SO}(3)$ is a curved surface sitting in $\mathbb{R}^{3\times3}$. At any point on it there is a flat plane that touches it, the **tangent space**, and a plane *is* a vector space. Calculus works there.

The curve, the plane touching it at $\bar{R}$, and $\exp$ carrying ξ back down onto it

At the identity that plane is called $\mathfrak{so}(3)$. Its members are the skew-symmetric matrices: three free numbers, no constraints between them.

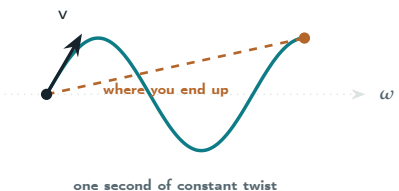
exp **and** Log

$\boldsymbol{\omega}$ says which axis to turn about, and how fast. exp turns that into an actual rotation: spin about that axis, at that rate, for one second, then stop. So $\boldsymbol{\omega} = (0,\ 0,\ \pi/2)$ is a turn about z at $\pi/2$ radians per second, and one second later you have turned 90°.



Log runs it backwards: give it a rotation, get back the axis and angle that made it. The length of $\mathrm{Log}(\mathrm{R}_1^\top \mathrm{R}_2)$ is the angle between two rotations, in radians, and that is what a rotation error is.

A twist



Six numbers, written $\xi = (\mathrm{v}, \omega)$: three of linear velocity, three of angular.

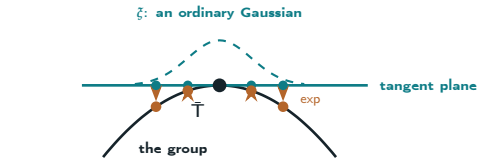
exp of a twist is where you end up after moving that way for one second, **turning while you translate**. The path is a helix, a screw.

So v is a velocity, not a displacement. The straight line from start to finish is shorter than the distance travelled and points somewhere else, which is why the translation you get out is not the v you put in.

> *A twist is a screw axis and a rate along it. A motion, not a destination.*

Uncertainty lives in the tangent space

A pose holds 6 degrees of freedom in 12 numbers, so a spread over those 12 numbers is meaningless: most directions leave the surface entirely. Put the **mean on the surface** and the **noise on the flat plane touching it there**.

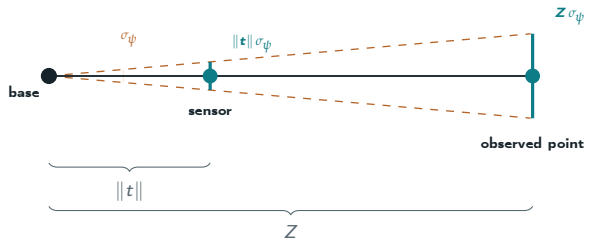


- On the plane it is an ordinary Gaussian over six numbers, so it has an ordinary 6×6 covariance. This is the number your SLAM system reports.
- Pushed down by exp it becomes a spread of poses, each one valid by construction.

Moving uncertainty between frames

the lever arm

An uncertainty is always quoted in some frame. Describe the same uncertainty from a different frame and it changes shape, because the conversion mixes **rotation into position**: a small angular error at the base becomes a position error out where the sensor is.



It grows with the mounting offset $\|t\|$, and then again with the range Z to whatever you observed. The same small angle, multiplied twice.

Half a degree, in centimetres

Base yaw error to sensor position error, then to point error

Half a degree at the base is seventeen centimetres at the far wall.

Chains, and the edge that drifts

A chain of estimated poses is the normal case: odometry, then a mount, then a calibration. Each link carries its own uncertainty into the final frame, and they accumulate.

same sensor, same noise, more distance travelled

- Every edge in the tree has a bounded error except world $\rightarrow$ base. That one integrates incremental motion, so it grows without limit.
- The error is in the integration, not the measurement. A better sensor does not fix it. Closing a loop does.

Quaternions, and which one to use when

This half opened by saying no choice of numbers escapes the problem. Quaternions are the choice you will be handed, so know what is inside one: a rotation stored as its axis n and its angle θ.

$$\mathrm{q} = \left(\mathrm{n} \sin \frac{\theta}{2},\ \cos \frac{\theta}{2} \right), \qquad \|\mathrm{q}\| = 1$$

- Three degrees of freedom, no angle at which they degenerate, cheap to compose, easy to renormalise. q and $-\mathrm{q}$ are the same rotation, so normalise the sign before comparing two.
- What they still do not give you is anywhere to differentiate. Four numbers with a constraint is no better than nine with six, and that is why the tangent space exists.

> *Store quaternions, do calculus in the tangent space, print matrices for humans.*

PnP, framed properly

Given 3D points **in a map you already have**, and their projections in the image you are looking at right now, find $[\mathrm{R} \mid \mathrm{t}]$. Both halves of today's lecture meet here.

$$\underset{\mathrm{R}\in\mathrm{SO}(3),\,\mathrm{t}\in\mathbb{R}^3}{\arg\min} \sum_i \left\| \mathrm{u}_i - \pi\left(\mathrm{K}\,[\mathrm{R} \mid \mathrm{t}]\,\tilde{\mathrm{X}}_i\right) \right\|^2$$

- The reprojection error from the first half, minimised over a pose instead of over P. As before, π is the perspective divide, $(a, b, c) \mapsto (a/c,\ b/c)$, turning the product back into a pixel.
- Six unknowns, two equations per correspondence, so **three points** is the minimum. Three points admit up to four valid solutions; a fourth disambiguates.

> *Wrapped in RANSAC, this is the answer to "where am I?"*

Why we optimise on the manifold

An optimiser repeatedly asks *which small motion would reduce the error*, then applies it. Both halves of that sentence are where today's second half arrives at once.

- **What it solves for** is a twist: six unconstrained numbers, rather than twelve matrix entries with six constraints between them. Unconstrained is what makes the step computable.
- **How it applies the answer** is by composing, not adding. The current estimate is moved along that twist, the way exp moves you along the surface.
- So the estimate is a valid pose at every iteration. Nothing to re-orthogonalise, nothing that slowly slides off the manifold.

> *Add a correction to a pose and you get something that is not a pose. Compose one and you always do.*

What you should be able to do

- Write down K for a sensor given its focal length and pixel pitch, and project a point through P.
- Say what the five intrinsic and six extrinsic numbers are, and which of them calibration has to estimate.
- Say why a pose cannot be corrected by addition, where the arithmetic happens instead, and what you get if you try it anyway.
- Say why a covariance on a pose is 6×6 and not 12×12, and what the lever arm does to it.
- Say what a twist is, and why moving along one for a second does not land you where its velocity was pointing.

Notes: Module **A1**, plus **A6** for the manifold material. Reference: the MathWorks calibration page under **Resources**, and Dellaert and Kaess, *Factor Graphs for Robot Perception*, §2.

Next time

L03 · Two views, correspondence, and robust estimation

One camera throws away depth. Two cameras get it back, but only after a harder problem: deciding which pixel in one image is the same point as which pixel in the other.

- Epipolar geometry, the essential and fundamental matrices, and the normalised eight-point algorithm, which is today's $Ap = 0$ again.
- Features and descriptors: what makes a point trackable.
- RANSAC, and why least squares is useless when 40% of your matches are wrong.
- Triangulation and bundle adjustment, the objective the whole field optimises.

No class Monday Sep 7, Labor Day. We next meet Wednesday Sep 9.